LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 – Abstract dan Interface



Enggar Tri Rahmadhani

103132400007

PS1DS – 05 - REG – 01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Abtract class

   Kelas yang tidak bisa di buat objeknya secara langsung. Kelas ini dibuat sebagai "cetak biru" untuk kelas turunannya. Kelas abstrak boleh memiliki method yang sudah ada isinya, dan juga boleh memiliki abstract method. Kelas turnan wajib mengisi method abstrak tersebut.

2. Interface

   Merupakan yang berisi daftar method tanpa isinya. Kelas ini memakai interface wajib mengimplementasikan semua method di dalamnya menggunakan keyword implements. Interface tidak punya konstruktor, dan semua atributnya bersifat konstanta.

3. Relasi realization

   Hubungan antar kelas dan interface yang diimplemantasikannya disebut realization. Artinya, kelas tersebut memenuhi semua kontrak method yang ada di interface.

4. Default method

   Adalah method di dalam interface yang sudah punya isi di tandai dengan keyword default. Agar kitab isa menambah method bar uke interface tanpa memaksa semua kelas yang sudah memakainya untuk menulis ulang kode.

5. Static method

   Merupakan method di dalam interface yang sudah punya isi, tapi tidak bisa di override oleh kelas implementasi.

6. Perbedaan abstract class dan interface

   - Isi method: abtract class boleh punya method berisi & method kosong. Interface hanya method kosong.
   - Atribut: abstract class boleh punya variable biasa. Interface hanya boleh konstanta.
   - Pewarisan: kelas hanya bisa extends satu abtract class, tapi bisa implements banyak interface.

B. Guided

1. gd1

file: Employee.java

```java
package com.enggar.guided.gd1;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh
diinstansiasi langsung
public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;

    }
    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji tapi rumusnya berbeda
    public abstract double earnings();
}
```

file: SalariedEmployee.java

```java
package com.enggar.guided.gd1;

// bentuk konkret
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

file: Main.java

```java
package com.enggar.guided.gd1;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("budi",
5000000);
        System.out.println("Nama : "+ salaried.getName());
        System.out.println("gaji : Rp" + salaried.earnings());
    }
}
```

Output :

```
ts\coolyeah\sems 5\pbo\modul07\t
Nama : budi
gaji : Rp5000000.0
PS C:\Users\ADVAN WORKPLUS\Docum
```

2. gd2

file: Employee.java

```java
package com.enggar.guided.gd2;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh
diinstansiasi langsung
public abstract class Employee implements Payable {
   private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus yang
berbeda-beda.
    public abstract double earnings();
```

```
    // saat ada penagihan, alihkan langsung ke metode kalkulasi gaji
(earning)
    @Override
    public double getPaymentAcount(){
        return earnings();
    }
}
```

file: Invoice.java

```
package com.enggar.guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getpartName(){
        return partName;
    }

    @Override
    public double getPaymentAcount(){
        return quantity * pricePerItem;
    }

}
```

file: Payable.java

```
package com.enggar.guided.gd2;

public interface Payable {
    double getPaymentAcount();
}
```

file: SalariedEmployee.java

```java
package com.enggar.guided.gd2;

// bentuk konkret
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

file: Main.java

```java
package com.enggar.guided.gd2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        // Dynamic dispatch : java otomatis mengeksekusi rumus yang
tepat sesuai objek

        for (Payable item: items){
            System.out.println("jumlah yang harus dibayar: Rp" +
item.getPaymentAcount());
        }
    }
}
```

Output :

```
in32-x86_64\bin\java.exe" "-XX:+ShowCodeDeta
ts\coolyeah\sems 5\pbo\modul07\target\classe
jumlah yang harus dibayar: Rp3000000.0
jumlah yang harus dibayar: Rp5000000.0
PS C:\Users\ADVAN WORKPLUS\Documents\coolyea
```

3. gd3

file: Employee.java

```java
package com.enggar.guided.gd3;

public abstract class Employee implements Payable {
      private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus yang
berbeda-beda.
    public abstract double earnings();

    // saat ada penagihan, alihkan langsung ke metode kalkulasi gaji
(earning)
    @Override
     public double getPaymentAcount(){
        return earnings();
    }
}
```

file: Invoice.java

```java
package com.enggar.guided.gd3;

public class Invoice implements Payable {
     private String partName;
    private int quantity;
```

```
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAcount(){
        return quantity * pricePerItem;
    }

}
```

file: Payable.java

```
package com.enggar.guided.gd3;

public interface Payable {
    double getPaymentAcount();

    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp " +
getPaymentAcount());
    }

    // fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG]" + message);
    }
}
```

file: SalariedEmployee.java

```
package com.enggar.guided.gd3;

public class SalariedEmployee extends  Employee {
```

```
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

file: Main.java

```
package com.enggar.guided.gd3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction(" Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");

    }
}
```

Output :

```
ts\coolyeah\sems 5\pbo\modul07\target\classes' 'com.
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp 4000000.0
Struk Resmi - Total: Rp 5000000.0
[AUDIT LOG]Transaksi selesai diproses.
PS C:\Users\ADVAN WORKPLUS\Documents\coolyeah\sems 5
```

C. Unguided

1. Soal Unguided – Modul 7

   Buatlah sebuah program Java untuk sistem kasir bioskop bernama Cinema POS yang menerapkan konsep abstract class, interface, dan polimorfisme. Program ini terdiri dari interface ItemTransaksi, abstract class TiketBioskop, dua kelas turunannya yaitu TiketReguler dan TiketVIP, satu kelas independen PaketSnack, dan kelas Main untuk menjalankan program. Semua atribut harus dibuat private dan diakses lewat getter, konstruktor wajib memakai keyword this, dan dilarang memakai ArrayList (gunakan array biasa).

   Pengerjaannya dibagi jadi empat tugas. Tugas pertama, buat abstract class TiketBioskop yang punya atribut judulFilm, hargaDasar, dan jumlahKursi, lengkap dengan dua abstract method hitungTotalBayar() dan getNamaTransaksi(). Kelas ini lalu diturunkan jadi TiketReguler (menambahkan nomorStudio, total bayar = harga dasar × jumlah kursi) dan TiketVIP (menambahkan biayaFasilitasVIP, total bayar = (harga dasar + biaya VIP) × jumlah kursi). Tugas kedua, buat interface ItemTransaksi berisi method getNominalBayar() dan getNamaTransaksi(), lalu implementasikan di TiketBioskop dan di kelas baru PaketSnack yang bukan turunan tiket. Tugas ketiga, tambahkan fitur Java 8 di interface, yaitu default method cetakRincianItem() untuk mencetak baris struk dan static method catatLogKasir() untuk mencetak log sistem. Tugas keempat, uji program di method main dengan membuat objek tiket reguler, tiket VIP, dan paket snack, simpan ketiganya ke dalam array ItemTransaksi[], lalu proses pakai perulangan untuk mencetak rincian dan menghitung total belanjaan.

Jawaban:

1. Unguided

   file: ItemTransaksi.java

```java
package com.enggar.unguided;

public interface ItemTransaksi {
    double getNominalBayar();

    String getNamaTransaksi();
```

```
    // Default Method
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp"
+ getNominalBayar());
    }

    // Static Method
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

file: PaketSnack.java

```
package com.enggar.unguided;

public class PaketSnack implements ItemTransaksi {
     private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket
                + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")";
    }
}
```

file: TiketBioskop.java

```java
package com.enggar.unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int
jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

file: TiketReguler.java

```java
package com.enggar.unguided;

public class TiketReguler extends TiketBioskop {
```

```
     private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int
jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " +
getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar()
+ ")";
    }
}
```

file: TiketVIP.java

```
    package com.enggar.unguided;

    public class TiketVIP extends TiketBioskop {
        private double biayaFasilitasVIP;

        public TiketVIP(String judulFilm, double hargaDasar, int
jumlahKursi, double biayaFasilitasVIP) {
            super(judulFilm, hargaDasar, jumlahKursi);
            this.biayaFasilitasVIP = biayaFasilitasVIP;
        }

        @Override
        public double hitungTotalBayar() {
            return (getHargaDasar() + biayaFasilitasVIP) *
getJumlahKursi();
        }

        @Override
```

```
        public String getNamaTransaksi() {
            return "Tiket VIP Premiere - " + getJudulFilm()
                    + " (" + getJumlahKursi() + " x Rp" +
(getHargaDasar() + biayaFasilitasVIP) + ")";
        }
    }
```

file: Main.java

```
package com.enggar.unguided;

public class Main {
    public static void main(String[] args) {

        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk
Pelanggan");

        ItemTransaksi tiketReguler = new TiketReguler("Interstellar",
40000, 2, 3);
        ItemTransaksi tiketVIP     = new TiketVIP("Dune: Part Two",
60000, 2, 25000);
        ItemTransaksi paketSnack   = new PaketSnack("Combo Popcorn
Caramel + 2 Soda", 1, 55000);

        ItemTransaksi[] keranjang = new ItemTransaksi[3];
        keranjang[0] = tiketReguler;
        keranjang[1] = tiketVIP;
        keranjang[2] = paketSnack;

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0.0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        System.out.println("----------------------------------------
--------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);
        System.out.println("========================================
====================");

        ItemTransaksi.catatLogKasir("Transaksi selesai diproses.");
    }
```

```
}
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi selesai diproses.
PS C:\Users\ADVAN WORKPLUS\Documents\coolyeah\sems 5\pbo>
```

Penjelasan :

Script ini membuat sistem kasir bioskop untuk menggabungkan tiket dan snack dalam satu transaksi memakai abstract class, interface, dan polimorfisme. Terdiri dari interface ItemTransaksi, abstract class TiketBioskop, kelas TiketReguler dan TiketVIP, kelas PaketSnack, dan Main.

Interface ItemTransaksi berisi getNominalBayar() dan getNamaTransaksi(), plus fitur Java 8: default method cetakRincianItem() dan static method catatLogKasir().

Abstract class TiketBioskop punya atribut judul film, harga dasar, dan jumlah kursi, serta mengisi getNominalBayar() dengan memanggil hitungTotalBayar(). TiketReguler menghitung hargaDasar * jumlahKursi, sedangkan TiketVIP memakai (hargaDasar + biayaVIP) * jumlahKursi. PaketSnack bukan turunan tiket, tapi tetap memakai interface yang sama dengan rumus jumlahPorsi * hargaPerPorsi.

Di Main, objek tiket reguler, tiket VIP, dan snack dimasukkan ke array ItemTransaksi[], lalu diproses dengan perulangan untuk mencetak rincian dan menghitung total. Hasil akhirnya grand total Rp305000.0

D. Kesimpulan

Kesimpulannya adalah modul 7 membahas abstract class sebagai kerangka dasar yang tidak bisa langsung dibuat objeknya dan harus diturunkan, serta interface sebagai kontrak method yang bisa dipakai banyak kelas sekaligus. Modul ini juga membahas fitur Java 8

berupa default method (method berisi di interface) dan static method (dipanggil lewat nama interface). Keduanya penting agar program OOP menjadi lebih fleksibel dan terstruktur.